

ICT 2105 Digital Logic Design

Prof. Dr. Muhammad Shahin Uddin

Email: shahin.ud din@mbstu.ac.bd

Lecture Outlines:

❖ Course Contents:

- This course provides an introduction to logic design and basic tools for the design of digital logic systems. A basic idea of number systems will be provided, followed by a discussion on combinational logic: logic gates, Boolean algebra, minimization techniques, arithmetic circuits (adders, subtractors), basic digital circuits (decoders, encoders, multiplexers, shift registers), programmable logic devices (PROM, PAL, PLA). The course will then cover sequential circuits: flip-flops, state transition tables and diagrams, state minimization, state machines, design of synchronous/asynchronous counters, RAM/ROM design. An introduction to programmable logic will also be provided. Hands-on experience will be provided through project on design of a sequential logic system. This course has separate mandatory laboratory session every week as CSE 231L.

Textbook:

Digital Design

Morris Mano and Michael D. Ciletti, 2012, 5th Edition Pearson Education, ISBN-13: 978-0-13-277420-8

Reference Books:

1. Digital Design: Principles and Practices, J F Wakerly, 4thed, Prentice Hall, 2005
2. Digital Logic Techniques, T J Stonham, 3rded, Chapman & Hall, 1996

Lecture Outlines:

❖ Course Objective:

- The objectives of this course are:
- To introduce different number system and their application
- To introduce Boolean logic operation and Boolean Algebra
- To teach students how to use Boolean Algebra and K-maps to realize two-level minimal/optimal combinational circuits
- To expose students in the introductory design process of combinational and sequential circuits
- To teach the operation of latches, flip-flops, counters and registers.
- To explain how to analyze and design sequential circuits built with various flip-flops.
- To understand the importance of state diagram representation of sequential circuits.
- To analyze and design clocked sequential circuits.
- To introduce using simulation tool for digital system design.

Digital Logic Design

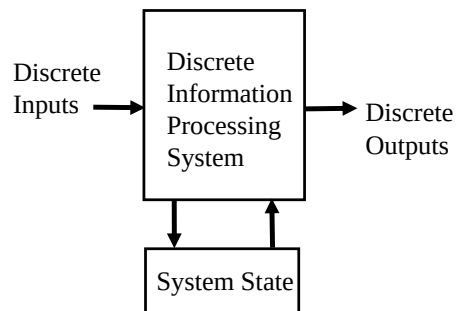
- Let me ask you few questions first.....(**prerequisite knowledge?**)
 - Did you take course on electrical circuits ??
 - Did you take course on electronics (Analog) ??
 - Do you know about signals (Analog/digital) ??

Digital Systems and Numbers

- Digital Systems
- Binary Numbers
- Number-base Conversions
- Octal and Hexadecimal Numbers
- Complements
- Signed Binary Numbers
- Binary Codes

Digital Systems

Takes a set of discrete information inputs and discrete internal information (system state) and generates a set of discrete information outputs.



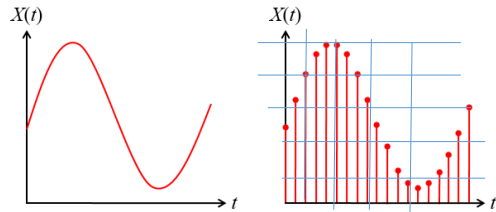
Digital Systems

❖ Application of Digital System

- | | |
|---------------------------------|--|
| ❖ Communication | ❖ Digital TV |
| ❖ Business transactions | ❖ Digital telephones |
| ❖ Traffic control | ❖ Digital televisions |
| ❖ Spacecraft guidance | ❖ Digital versatile discs |
| ❖ Medical treatment | ❖ Digital cameras |
| ❖ Weather monitoring | ❖ Handheld devices, and |
| ❖ Internet | ❖ Digital computers |
| ❖ Telephone switching exchanges | ❖ Many other commercial, industrial, and |
| ❖ Digital camera | scientific enterprises. |
| ❖ Electronic calculators, PDA's | |

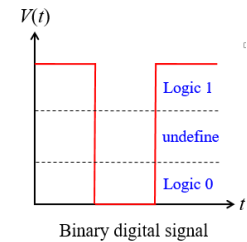
Analog and Digital Signal

- ❖ Analog system
 - The physical quantities or signals may vary continuously over a specified range.
- ❖ Digital system
 - The physical quantities or signals can assume only discrete values.
 - Greater accuracy

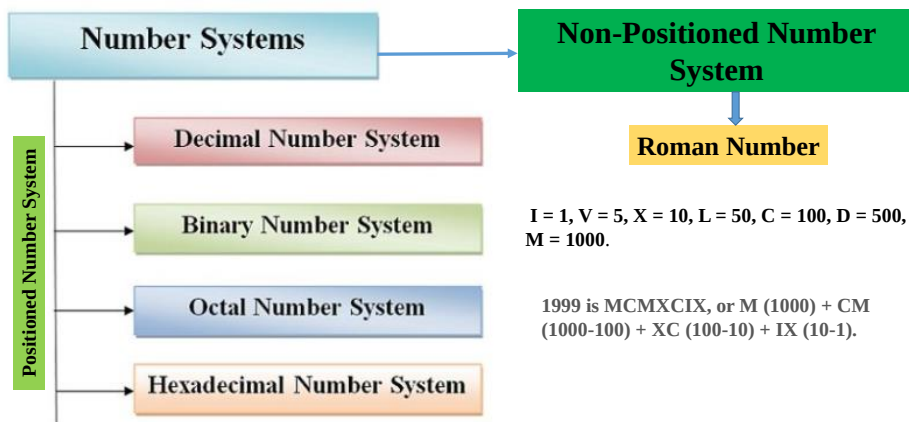


Binary Digital Signal

- ❖ An information variable represented by physical quantity.
- ❖ For digital systems, the variable takes on discrete values.
 - Two level, or binary values are the most prevalent values.
- ❖ Binary values are represented abstractly by:
 - Digits 0 and 1
 - Words (symbols) False (F) and True (T)
 - Words (symbols) Low (L) and High (H)
 - And words on and off
- ❖ Binary values are represented by values or ranges of values of physical quantities.

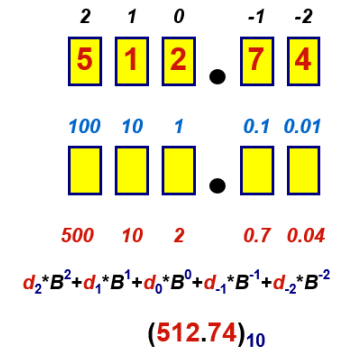


Number System



Decimal Number System

- ❖ Base (also called radix) = 10
 - 10 digits { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }
- ❖ Digit Position
 - Integer & fraction
- ❖ Digit Weight
 - Weight = $(Base)^{Position}$
- ❖ Magnitude
 - Sum of "Digit x Weight"
- ❖ Formal Notation



Octal Number System

- ❖ Base = 8
 - 8 digits { 0, 1, 2, 3, 4, 5, 6, 7 }
- ❖ Weights
 - Weight = $(Base)^{Position}$
- ❖ Magnitude
 - Sum of “Digit x Weight”
- ❖ Formal Notation

$$\begin{array}{ccccccc}
 & 64 & 8 & 1 & & 1/8 & 1/64 \\
 & \boxed{5} & \boxed{1} & \boxed{2} & \bullet & \boxed{7} & \boxed{4} \\
 & 2 & 1 & 0 & & -1 & -2 \\
 \hline
 & 5 \cdot 8^2 + 1 \cdot 8^1 + 2 \cdot 8^0 + 7 \cdot 8^{-1} + 4 \cdot 8^{-2} \\
 \hline
 & = (330.9375)_{10} \\
 & = (512.74)_8
 \end{array}$$

Binary Number System

- ❖ Base = 2
 - 2 digits { 0, 1 }, called *binary digits* or “*bits*”
- ❖ Weights
 - Weight = $(Base)^{Position}$
- ❖ Magnitude
 - Sum of “Bit x Weight”
- ❖ Formal Notation
- ❖ Groups of bits
 - 8 bits = Byte

$$\begin{array}{ccccccc}
 & 4 & 2 & 1 & & 1/2 & 1/4 \\
 & \boxed{1} & \boxed{0} & \boxed{1} & \bullet & \boxed{0} & \boxed{1} \\
 & 2 & 1 & 0 & & -1 & -2 \\
 \hline
 & 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 0 \cdot 2^{-1} + 1 \cdot 2^{-2} \\
 \hline
 & = (5.25)_{10} \\
 & = (101.01)_2 \\
 \hline
 & \boxed{1011} \\
 & \boxed{11000101}
 \end{array}$$

Binary Number

Powers of Two

<i>n</i>	2^n	<i>n</i>	2^n	<i>n</i>	2^n
0	1	8	256	16	65,536
1	2	9	512	17	131,072
2	4	10	1,024 (1K)	18	262,144
3	8	11	2,048	19	524,288
4	16	12	4,096 (4K)	20	1,048,576 (1M)
5	32	13	8,192	21	2,097,152
6	64	14	16,384	22	4,194,304
7	128	15	32,768	23	8,388,608

Hexadecimal Number System

- ❖ Base = 16
 - 16 digits { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F }
- ❖ Weights
 - Weight = $(Base)^{Position}$
- ❖ Magnitude
 - Sum of “Digit x Weight”
- ❖ Formal Notation

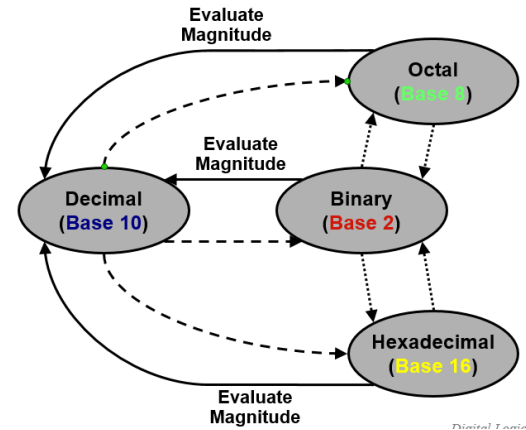
$$\begin{array}{ccccccc}
 & 256 & 16 & 1 & & 1/16 & 1/256 \\
 & \boxed{1} & \boxed{E} & \boxed{5} & \bullet & \boxed{7} & \boxed{A} \\
 & 2 & 1 & 0 & & -1 & -2 \\
 \hline
 & 1 \cdot 16^2 + 14 \cdot 16^1 + 5 \cdot 16^0 + 7 \cdot 16^{-1} + 10 \cdot 16^{-2} \\
 \hline
 & = (485.4765625)_{10} \\
 & = (1E5.7A)_{16}
 \end{array}$$

The Power of 2

n	2 ⁿ	
0	2 ⁰ =1	
1	2 ¹ =2	
2	2 ² =4	
3	2 ³ =8	
4	2 ⁴ =16	
5	2 ⁵ =32	
6	2 ⁶ =64	
7	2 ⁷ =128	

n	2 ⁿ	
8	2 ⁸ =256	
9	2 ⁹ =512	
10	2 ¹⁰ =1024	Kilo
11	2 ¹¹ =2048	
12	2 ¹² =4096	
20	2 ²⁰ =1M	Mega
30	2 ³⁰ =1G	Giga
40	2 ⁴⁰ =1T	Tera

Number Base Conversions



Decimal (*Integer*) to Binary Conversion

- ❖ Divide the number by the 'Base' (=2)
- ❖ Take the remainder (either 0 or 1) as a coefficient
- ❖ Take the quotient and repeat the division
- ❖ Example:

(13)₁₀

	Quotient	Remainder	Coefficient
13 / 2 =	6	1	a ₀ = 1
6 / 2 =	3	0	a ₁ = 0
3 / 2 =	1	1	a ₂ = 1
1 / 2 =	0	1	a ₃ = 1

Answer: **(13)₁₀ = (a₃ a₂ a₁ a₀)₂ = (1101)₂**

↑
↑
 MSB LSB

Decimal (*Fraction*) to Binary Conversion

- ❖ Multiply the number by the 'Base' (=2)
- ❖ Take the integer (either 0 or 1) as a coefficient
- ❖ Take the resultant fraction and repeat the division
- ❖ Example: **(0.625)₁₀**

	Integer	Fraction	Coefficient
0.625 * 2 =	1	. 25	a ₁ = 1
0.25 * 2 =	0	. 5	a ₂ = 0
0.5 * 2 =	1	. 0	a ₃ = 1

Answer: **(0.625)₁₀ = (0.a₁ a₂ a₃)₂ = (0.101)₂**

↑
↑
 MSB LSB

Decimal (*Fraction*) to Binary Conversion

- ❖ Multiply the number by the 'Base' (=2)
- ❖ Take the integer (either 0 or 1) as a coefficient
- ❖ Take the resultant fraction and repeat the division
- ❖ Example:

(0.625)₁₀

	Integer	Fraction	Coefficient
0.625 * 2 =	1	. 25	a ₋₁ = 1
0.25 * 2 =	0	. 5	a ₋₂ = 0
0.5 * 2 =	1	. 0	a ₋₃ = 1

Answer: (0.625)₁₀ = (0.a₋₁a₋₂a₋₃)₂ = (0.101)₂

↑
↑
MSB
LSB

Decimal to Octal Conversion

Example: (175)₁₀

	Quotient	Remainder	Coefficient
175 / 8 =	21	7	a ₀ = 7
21 / 8 =	2	5	a ₁ = 5
2 / 8 =	0	2	a ₂ = 2

Answer: (175)₁₀ = (a₂a₁a₀)₈ = (257)₈

Example: (0.3125)₁₀

	Integer	Fraction	Coefficient
0.3125 * 8 =	2	. 5	a ₋₁ = 2
0.5 * 8 =	4	. 0	a ₋₂ = 4

Answer: (0.3125)₁₀ = (0.a₋₁a₋₂)₈ = (0.24)₈

Binary– Octal Conversion

- ▣ 8 = 2³
- ▣ Each group of 3 bits represents an octal digit

Example:

Assume Zeros

(1 0 1 1 0 . 0 1)₂

┌───┬───┬───┐

(2 6 . 2)₈

Octal	Binary
0	0 0 0
1	0 0 1
2	0 1 0
3	0 1 1
4	1 0 0
5	1 0 1
6	1 1 0
7	1 1 1

Works **both** ways (Binary to Octal & Octal to Binary)

Binary–Hexadecimal Conversion

▣ 16 = 2⁴

- ▣ Each group of 4 bits represents a hexadecimal digit

Example:

Assume Zeros

(1 0 1 1 0 . 0 1)₂

┌───┬───┬───┐

(1 6 . 4)₁₆

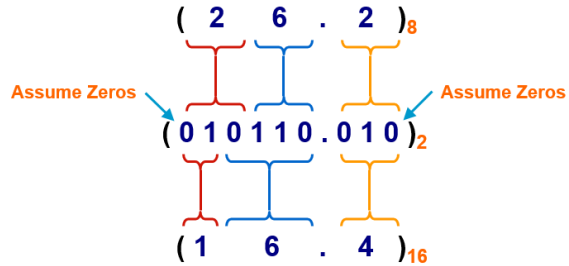
Hex	Binary
0	0 0 0 0
1	0 0 0 1
2	0 0 1 0
3	0 0 1 1
4	0 1 0 0
5	0 1 0 1
6	0 1 1 0
7	0 1 1 1
8	1 0 0 0
9	1 0 0 1
A	1 0 1 0
B	1 0 1 1
C	1 1 0 0
D	1 1 0 1
E	1 1 1 0
F	1 1 1 1

Works **both** ways (Binary to Hex & Hex to Binary)

Octal–Hexadecimal Conversion

Convert to **Binary** as an intermediate step

Example:



Works **both** ways (*Octal to Hex & Hex to Octal*)

Decimal, Binary, Octal and Hexadecimal

Decimal	Binary	Octal	Hex
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Addition

$$\begin{array}{r}
 11 \quad \leftarrow \text{Carry} \\
 55 \\
 + 55 \\
 \hline
 110
 \end{array}$$

Binary Addition

$$\begin{array}{r}
 111111 \\
 111101 = 61 \\
 + 110111 = 23 \\
 \hline
 1010100 = 84
 \end{array}$$

Binary Subtraction

Borrow a “Base” when needed

$$\begin{array}{r}
 12000 \\
 \cancel{0}20\cancel{0}\cancel{0}1 \\
 \cancel{1}0\cancel{1}\cancel{1}01 \\
 - 10111 \\
 \hline
 0110110
 \end{array}
 \begin{array}{l}
 = (10)_2 \\
 = 77 \\
 = 23 \\
 = 54
 \end{array}$$

Binary Multiplication

Bit by bit

$$\begin{array}{cccccc} & & & 1 & 0 & 1 & 1 & 1 \\ \mathbf{x} & & & & 1 & 0 & 1 & 0 \\ \hline & & & 0 & 0 & 0 & 0 & 0 \\ & & 1 & 0 & 1 & 1 & 1 & \\ & 0 & 0 & 0 & 0 & 0 & & \\ 1 & 0 & 1 & 1 & 1 & & & \\ \hline 1 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{array}$$

Base Conversion

- **Examples:**

- $(11010)_2 = (?)_{10}$

- $(627)_8 = (?)_{10}$

Base Conversion

- $(11010)_2 \rightarrow (?)_{10}$

- $(627)_8 \rightarrow (?)_{10}$

Examples of Base Conversion (3)

• $(315)_{10} = (473)_8$

• $(315)_{10} = (13B)_{16}$

Example 1
Convert decimal 41 to binary.

Base Conversion (5)

• $(0.479)_{10} = (0.3651...)_{8}$
MSD $3.832 \leftarrow 0.479 \times 8$
 $6.656 \leftarrow 0.832 \times 8$
 $5.248 \leftarrow 0.656 \times 8$
LSD $1.984 \leftarrow 0.248 \times 8$
 ...

• $(0.479)_{10} = (0.0111...)_{2}$
MSD $0.9580 \leftarrow 0.479 \times 2$
 $1.9160 \leftarrow 0.9580 \times 2$
 $1.8320 \leftarrow 0.9160 \times 2$
LSD $1.6640 \leftarrow 0.8320 \times 2$
 ...

Example 1
Convert decimal 41 to binary.

	Integer Quotient		Remainder	Coefficient
$41/2 =$	20	+	$\frac{1}{2}$	$a_0 = 1$
$20/2 =$	10	+	0	$a_1 = 0$
$10/2 =$	5	+	0	$a_2 = 0$
$5/2 =$	2	+	$\frac{1}{2}$	$a_3 = 1$
$2/2 =$	1	+	0	$a_4 = 0$
$1/2 =$	0	+	$\frac{1}{2}$	$a_5 = 1$

Therefore, the answer is $(41)_{10} = (a_5a_4a_3a_2a_1a_0)_2 = (101001)_2$.

Example 1

Convert decimal 41 to binary.

The arithmetic process can be manipulated more conveniently as follows:

Integer	Remainder
41	
20	1
10	0
5	0
2	1
1	0
0	1 101001 = answer

Conversion from decimal integers to any base- r system is similar to this example, except that division is done by r instead of 2.

Example 2

Convert decimal 153 to octal. The required base r is 8. First, 153 is divided by 8 to give an integer quotient of 19 and a remainder of 1. Then 19 is divided by 8 to give an integer quotient of 2 and a remainder of 3. Finally, 2 is divided by 8 to give a quotient of 0 and a remainder of 2. This process can be conveniently manipulated as follows:

Example 2

Convert decimal 153 to octal. The required base r is 8. First, 153 is divided by 8 to give an integer quotient of 19 and a remainder of 1. Then 19 is divided by 8 to give an integer quotient of 2 and a remainder of 3. Finally, 2 is divided by 8 to give a quotient of 0 and a remainder of 2. This process can be conveniently manipulated as follows:

153	
19	1
2	3
0	2 = (231) ₈

The conversion of a decimal *fraction* to binary is accomplished by a method similar to that used for integers. However, multiplication is used instead of division, and integers instead of remainders are accumulated. Again, the method is best explained by example.

Complements

- ❖ r 's complements
- ❖ $(r-1)$'s complements

	r 's	$(r-1)$'s
$r=10$	10's com	9's
$r=2$	2's com	1's
$r=8$	8's com	7's
$r=16$	16's com	15's

Complements

- ❖ Complement number system:
 - An alternative representation of numbers in a fixed-radix number system.
- ❖ Why complement of a number is used?
 - Simplify the subtraction operation
 - Logical manipulations.
- ❖ For each radix- r system (radix r represents base of number system) there are two types of complements.

Binary Subtraction using 1's complement

- ❖ Step 1: Convert number to be subtracted to its 1's complement.
- ❖ Step 2: Perform addition
- ❖ If the final carry is 1 then add it to the results obtained in step 2 is negative in step 2
- ❖ Example: Perform $(1100)_2 - (0101)_2$

$A = 1100$ and $B = 0101$

1's complement of $B = 1010 = -B$

$$\begin{array}{r}
 1100 \\
 1010 \\
 \hline
 \textcircled{1}0110 \\
 \textcircled{1} \\
 \hline
 0111
 \end{array}$$

Complements

- ❖ There are two types of complements for each base- r system: the radix complement and diminished radix complement.
- ❖ **Diminished Radix Complement - $(r-1)$'s Complement**
 - Given a number N in base r having n digits, the $(r-1)$'s complement of N is defined as:

$$(r^n - 1) - N$$
- ❖ **Example for 6-digit decimal numbers:**
 - 9's complement is $(r^n - 1) - N = (10^6 - 1) - N = 999999 - N$
 - 9's complement of 546700 is $999999 - 546700 = 453299$
- ❖ **Example for 7-digit binary numbers:**
 - 1's complement is $(r^n - 1) - N = (2^7 - 1) - N = 1111111 - N$
 - 1's complement of 1011000 is $1111111 - 1011000 = 0100111$
- ❖ **Observation:**
 - Subtraction from $(r^n - 1)$ will never require a borrow
 - Diminished radix complement can be computed digit-by-digit
 - For binary: $1 - 0 = 1$ and $1 - 1 = 0$

Complements

1's Complement (*Diminished Radix Complement*)

- ♦ All '0's become '1's
 - ♦ All '1's become '0's
- Example $(10110000)_2$
 $\Rightarrow (01001111)_2$

If you add a number and its 1's complement ...

$$\begin{array}{r}
 10110000 \\
 + 01001111 \\
 \hline
 11111111
 \end{array}$$

Complements

Radix Complement:

The r 's complement of an n digit number in base r is defined as $(r^n - N)$ for $N \neq 0$ and as 0 for $N = 0$. Comparing the $(r-1)$'s complement, we note that the r 's complement is obtained by adding 1 to the $(r-1)$'s complement, since $(r^n - N) = [(r^n - 1) - N] + 1$.

Example: Base-10

The 10's complement of 012398 is 987602
The 10's complement of 246700 is 753300

Example: Base-2

The 2's complement of 1101100 is 0010100
The 2's complement of 0110111 is 1001001

Complements

Radix Complement:

The r 's complement of an n digit number in base r is defined as $(r^n - N)$ for $N \neq 0$ and as 0 for $N = 0$. Comparing the $(r-1)$'s complement, we note that the r 's complement is obtained by adding 1 to the $(r-1)$'s complement, since $(r^n - N) = [(r^n - 1) - N] + 1$.

Example: Base-10

The 10's complement of 012398 is 987602
The 10's complement of 246700 is 753300

$1000000 - 1 = 9999999$
 $999999 -$
 $012398 = 987601 + 1$

Example: Base-2

The 2's complement of 1101100 is 0010100
The 2's complement of 0110111 is 1001001

$$\begin{array}{r} 1\ 1\ 1\ 1\ 1\ 1\ 1 \\ - 1\ 1\ 0\ 1\ 1\ 0\ 0 \\ \hline 0\ 0\ 1\ 0\ 1\ 0\ 0 \\ + \qquad\qquad\qquad 1 \\ \hline 0\ 0\ 1\ 0\ 1\ 0\ 0 \end{array}$$

Complements

2's Complement (Radix Complement)

♦ Take 1's complement then add 1

OR ♦ Toggle all bits to the left of the first '1' from the right

Example:

Number:

1's Comp.:

$\begin{array}{r} 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\ 0\ 1\ 0\ 0\ 1\ 1\ 1\ 1 \\ + \qquad\qquad\qquad 1 \\ \hline 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0 \end{array}$	$\begin{array}{r} 1\ 0\ 1\ 1\ 0\ 0\ 0\ 0 \\ 0\ 1\ 0\ 1\ 0\ 0\ 0\ 0 \end{array}$
--	---

Complements

❖ Subtraction with Complements

- The subtraction of two n -digit unsigned numbers $M - N$ in base r can be done as follows:

1. Add the minuend M to the r 's complement of the subtrahend N . Mathematically, $M + (r^n - N) = M - N + r^n$.
2. If $M \geq N$, the sum will produce an end carry r^n , which can be discarded; what is left is the result $M - N$.
3. If $M < N$, the sum does not produce an end carry and is equal to $r^n - (N - M)$, which is the r 's complement of $(N - M)$. To obtain the answer in a familiar form, take the r 's complement of the sum and place a negative sign in front.

Complements

Example

- Given the two binary numbers $X = 1010100$ and $Y = 1000011$, perform the subtraction (a) $X - Y$; and (b) $Y - X$, by using 2's complement.

$$\begin{array}{r}
 \text{(a)} \quad X = 1010100 \\
 \text{2's complement of } Y = +0111101 \\
 \hline
 \text{Sum} = 10010001 \\
 \text{Discard end carry } 2^7 = -10000000 \\
 \hline
 \text{Answer. } X - Y = 0010001
 \end{array}$$

$$\begin{array}{r}
 \text{(b)} \quad Y = 1000011 \\
 \text{2's complement of } X = +0101100 \\
 \hline
 \text{Sum} = 1101111
 \end{array}$$

There is no end carry.
Therefore, the answer is
 $Y - X = -$ (2's complement
of 1101111) = - 0010001.

Complements

Subtraction of unsigned numbers can also be done by means of the $(r - 1)$'s complement. Remember that the $(r - 1)$'s complement is one less than the r 's complement.

Example

- Repeat Example 1.7, but this time using 1's complement.

$$\begin{array}{r}
 \text{(a)} \quad X - Y = 1010100 - 1000011 \\
 X = 1010100 \\
 \text{1's complement of } Y = +0111100 \\
 \hline
 \text{Sum} = 10010000 \\
 \text{End-around carry} = +1 \\
 \hline
 \text{Answer. } X - Y = 0010001
 \end{array}$$

$$\begin{array}{r}
 \text{(b)} \quad Y - X = 1000011 - 1010100 \\
 Y = 1000011 \\
 \text{1's complement of } X = +0101101 \\
 \hline
 \text{Sum} = 1101110
 \end{array}$$

There is no end carry.
Therefore, the answer is $Y - X = -$ (1's complement of
1101110) = - 0010001.

Signed Binary Numbers

To represent negative integers, we need a notation for negative values.

It is customary to represent the sign with a bit placed in the leftmost position of the number since binary digits.

The convention is to make the **sign bit 0 for positive and 1 for negative**.

Example:

Signed-magnitude representation:	10001001
Signed-1's-complement representation:	11110110
Signed-2's-complement representation:	11110111

Table lists all possible four-bit signed binary numbers in the three representations.

Signed Binary Numbers

Signed Binary Numbers

Decimal	Signed-2's Complement	Signed-1's Complement	Signed Magnitude
+7	0111	0111	0111
+6	0110	0110	0110
+5	0101	0101	0101
+4	0100	0100	0100
+3	0011	0011	0011
+2	0010	0010	0010
+1	0001	0001	0001
+0	0000	0000	0000
-0	—	1111	1000
-1	1111	1110	1001
-2	1110	1101	1010
-3	1101	1100	1011
-4	1100	1011	1100
-5	1011	1010	1101
-6	1010	1001	1110
-7	1001	1000	1111
-8	1000	—	—

Signed Binary Numbers

Arithmetic addition

- ♦ The addition of two numbers in the signed-magnitude system follows the rules of ordinary arithmetic. **If the signs are the same, we add the two magnitudes and give the sum the common sign. If the signs are different, we subtract the smaller magnitude from the larger and give the difference the sign of the larger magnitude.**
- ♦ The addition of two signed binary numbers with negative numbers represented in signed-2's-complement form is obtained from the addition of the two numbers, including their sign bits.
- ♦ A carry out of the sign-bit position is discarded.

Example:

+ 6	00000110	- 6	11111010
+ 13	00001101	+ 13	00001101
+ 19	00010011	+ 7	00000111
+ 6	00000110	- 6	11111010
- 13	11110011	- 13	11110011
- 7	11111001	- 19	11101101

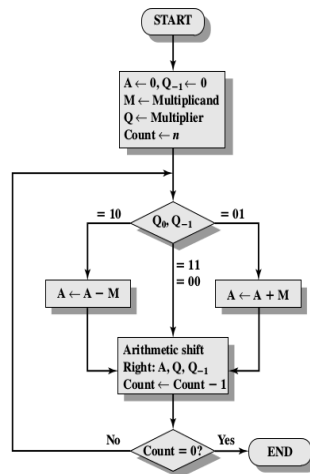
Booth's Algorithm for 2's Complement Multiplication

Can Multiply

- ❖ Two positive numbers
- ❖ Two negative numbers
- ❖ One positive and one negative number

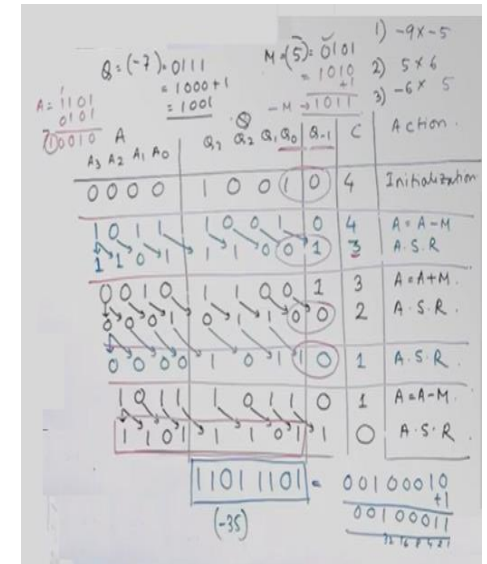
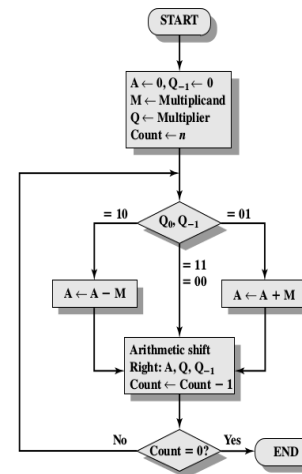
- ✓ Important part of ALU design
- ✓ Use adder circuit only
- ✓ No required multiplier circuit

Booth's Algorithm for 2's Complement Multiplication



A	Q	Q _{n-1}	M	
0000	0011	0	0111	Initial values
1001	0011	0	0111	A ← A - M
1100	1001	1	0111	Shift
1110	0100	1	0111	Shift
0101	0100	1	0111	A ← A + M
0010	1010	0	0111	Shift
0001	0101	0	0111	Shift

Booth's Multiplication



Binary Codes

BCD Code

- ◆ In this **code** each decimal digit is represented by a 4-bit binary **number**. **BCD** is a way to express each of the decimal digits with a binary **code**
- ◆ A number with k decimal digits will require 4k bits in BCD.
- ◆ Decimal 396 is represented in BCD with 12bits as 0011 1001 0110, with each group of 4 bits representing one decimal digit.
- ◆ The binary combinations 1010 through 1111 are not used and have no meaning in BCD.

Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Binary Code

Example:

- ◆ Consider decimal 185 and its corresponding value in BCD and binary:

$$(185)_{10} = (0001\ 1000\ 0101)_{BCD} = (10111001)_2$$

BCD addition

4	0100	4	0100	8	1000
+ 5	+ 0101	+ 8	+ 1000	+ 9	+ 1001
9	1001	12	1100	17	10001
		+ 0110		+ 0110	
		10010		10111	

Binary Code

Example:

- ◆ Consider the addition of $184 + 576 = 760$ in BCD:

BCD	1	1		
	0001	1000	0100	184
	+ 0101	0111	0110	+576
Binary sum	0111	10000	1010	
Add 6	—	0110	0110	—
BCD sum	0111	0110	0000	760

Binary Codes

- ❖ Other Decimal Codes – weighted representation of decimal numbers in binary.

Four Different Binary Codes for the Decimal Digits

Decimal Digit	BCD 8421	2421	Excess-3	8, 4, -2, -1
0	0000	0000	0011	0000
1	0001	0001	0100	0111
2	0010	0010	0101	0110
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1011
6	0110	1100	1001	1010
7	0111	1101	1010	1001
8	1000	1110	1011	1000
9	1001	1111	1100	1111
Unused bit	1010	0101	0000	0001
	1011	0110	0001	0010
	1100	0111	0010	0011
combinations	1101	1000	1101	1100
	1110	1001	1110	1101
	1111	1010	1111	1110

Gray Code

- ❖ **Definition:** Gray Code is the minimum-change code category of coding in which, the two consecutive values changes by only a single bit. More specifically we can say, it is a [binary number system](#) where while moving from one step to the next, only a single bit shows variation.
- ❖ The advantage is that only bit in the code group changes in going from one number to the next.
 - Error detection.
 - Representation of analog data.
 - Low power design

Gray Code

The conversion process from binary code to a gray involves the following steps:

- ❖ Firstly, record the most significant bit or MSB or the leftmost bit of the given binary data as it is, to have MSB of gray equivalent.
- ❖ Now, proceed towards adding the adjacent bits of the binary data starting from MSB with its adjacent bit to LSB. While adding, put the summation obtained in place of next bit and ignoring the carry.
- ❖ Repeat the same process for all the bits in the sequence till LSB.
- ❖ This is how the binary code is converted into gray equivalent.

Decimal Value	Binary Code	Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
A	1010	1111

BCD-to-Excess-3 Code converter

- ❖ BCD is a code for the decimal digits 0-9
- ❖ Excess-3 is also a code for the decimal digits

Decimal Digit	Input BCD	Output Excess-3
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

❑ Specification of BCD-to-Excess3

- ❖ **Inputs:** a BCD input, A,B,C,D with A as the most significant bit and D as the least significant bit.
- ❖ **Outputs:** an Excess-3 output W,X,Y,Z that corresponds to the BCD input.
- ❖ **Internal operation** – circuit to do the conversion in combinational logic.

❑ Formulation of BCD-to-Excess-3

- ❖ Excess-3 code is easily formed by adding a binary 3 to the binary or BCD for the digit.
- ❖ There are 16 possible inputs for both BCD and Excess-3.
- ❖ It can be assumed that only valid BCD inputs will appear so the six combinations not used can be treated as don't cares.

ASCII Code

- ❖ **ASCII** is the acronym for the American Standard **Code** for Information Interchange. It is a **code** for representing 128 English **characters** as numbers, with each letter assigned a number from 0 to 127. For **example**, the **ASCII code** for uppercase M is 77.
- ❖ **ASCII**, a standard data-transmission **code** that is **used by** smaller and less-powerful computers to represent both textual data (letters, numbers, and punctuation marks) and noninput-device commands (control **characters**).

ASCII Code

- American Standard Code for Information Interchange (ASCII) Character Code

American Standard Code for Information Interchange (ASCII)

$b_4b_3b_2b_1$	$b_7b_6b_5$							
	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	p
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	.	<	L	\	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	US	/	?	O	_	o	DEL

ASCII Code

ASCII Character Code

Control characters			
NUL	Null	DLE	Data-link escape
SOH	Start of heading	DC1	Device control 1
STX	Start of text	DC2	Device control 2
ETX	End of text	DC3	Device control 3
EOT	End of transmission	DC4	Device control 4
ENQ	Enquiry	NAK	Negative acknowledge
ACK	Acknowledge	SYN	Synchronous idle
BEL	Bell	ETB	End-of-transmission block
BS	Backspace	CAN	Cancel
HT	Horizontal tab	EM	End of medium
LF	Line feed	SUB	Substitute
VT	Vertical tab	ESC	Escape
FF	Form feed	FS	File separator
CR	Carriage return	GS	Group separator
SO	Shift out	RS	Record separator
SI	Shift in	US	Unit separator
SP	Space	DEL	Delete

ASCII Character Codes

- ❖ American Standard Code for Information Interchange
- ❖ A popular code used to represent information sent as character-based data.
- ❖ It uses 7-bits to represent:
 - 94 Graphic printing characters.
 - 34 Non-printing characters.
- ❖ Some non-printing characters are used for text format (e.g. BS = Backspace, CR = carriage return).

ASCII Properties

- ❖ ASCII has some interesting properties:
 - Digits 0 to 9 span Hexadecimal values 30_{16} to 39_{16}
 - Upper case A-Z span 41_{16} to $5A_{16}$
 - Lower case a-z span 61_{16} to $7A_{16}$
 - Lower to upper case translation (and vice versa) occurs by flipping **bit 6**.

Extended ASCII Code/ Error Detection Code

- ❖ To detect errors in data communication and processing, an eighth bit is sometimes added to the ASCII character to indicate its parity.
- ❖ A parity bit is an extra bit included with a message to make the total number of 1's either even or odd.
- ❖ Consider the following two characters and their even and odd parity:

	With even parity	With odd parity
ASCII A = 1000001	01000001	11000001
ASCII T = 1010100	11010100	01010100

- ❖ The parity bit is helpful in detecting errors during the transmission of information.
- ❖ Handled by generating an even parity bit at the sending end for each character.
- ❖ The eight-bit characters that include parity bits are transmitted to their destination.
- ❖ The parity of each character is then checked at the receiving end.
- ❖ If the parity of the received character is not even, then at least one bit has changed value during the transmission.

Binary Codes

- ❖ In the coding, when numbers, letters or words are represented by a specific group of symbols, it is said that the number, letter or word is being encoded.
- ❖ The group of symbols is called as a code.
- ❖ The digital data is represented, stored and transmitted as group of binary bits.
- ❖ This group is also called as binary code. The binary code is represented by the number as well as alphanumeric letter.

Advantages of Binary Code

- ❖ Binary codes are suitable for the computer applications.
- ❖ Binary codes are suitable for the digital communications.
- ❖ Binary codes make the analysis and designing of digital circuits if we use the binary codes.
- ❖ Since only 0 & 1 are being used, implementation becomes easy.

Classification of binary codes

The codes are broadly categorized into following four categories.

- Weighted Codes
- Non-Weighted Codes
- Binary Coded Decimal Code
- Alphanumeric Codes
- Error Detecting Codes
- Error Correcting Codes

Weighted Codes

- ❖ Weighted binary codes are those binary codes which obey the positional weight principle. Each position of the number represents a specific weight. Several systems of the codes are used to express the decimal digits 0 through 9. In these codes each decimal digit is represented by a group of four bits.

Non-Weighted Codes

- ❖ In this type of binary codes, the positional weights are not assigned. The examples of non-weighted codes are **Excess-3** code and **Gray code**.

EXCESS-3 CODE

The Excess-3 code is also called as XS-3 code. It is non-weighted code used to express decimal numbers. The Excess-3 code words are derived from the 8421 BCD code words adding $(0011)_2$ or $(3)_{10}$ to each code word in 8421. The excess-3 codes are obtained as follows

THANKS